CO21BTECH11002
Aayush Kumar
Theory Assignment 3

Q1 (4.1)

$s_1 = w_1(x)r_2(y)r_1(x)c_1r_2(x)w_2(y)c_2$

2PL:

There is no waiting by any thread due to y since it is accessed by only T_2.
Since T_1 releases the lock on x before T_2 requests it (T_1 commits before $r_2(x)$), there is no waiting due to x. Hence, the operations are performed in the order in which they arrive. Hence, final output:

$wl_1(x)w_1(x)rl_2(y)r_2(y)r_1(x)wu_1(x)c_1rl_2(x)r_2(x)wl_2(y)w_2(y)wu_2(y)ru_2(x)c_2$

O2PL:

Following the same arguments as in O2PL, the operations are performed in the order in which they arrive. Hence, final output:

$wl_1(x)w_1(x)rl_2(y)r_2(y)r_1(x)wu_1(x)c_1rl_2(x)r_2(x)wl_2(y)w_2(y)wu_2(y)ru_2(x)c_2$

BTO:

$ts(t_1) < ts(t_2)$
The only conflicting operations are $w_1(x)$, $r_2(x)$. But since $w_1(x)$ comes before $r_2(x)$, both can be executed in the order they arrive. Hence, final output:

$w_1(x)r_2(y)r_1(x)c_1r_2(x)w_2(y)c_2$

SGT:

The only conflicting operations are $w_1(x)$, $r_2(x)$. Hence, there are no cycles and both transactions will execute without aborting. Hence, final output:

$w_1(x)r_2(y)r_1(x)c_1r_2(x)w_2(y)c_2$

$s_2 = r_1(x)r_2(x)w_3(x)w_4(x)w_1(x)c_1w_2(x)c_2c_3c_4$

2PL:

Both T_1 and T_2 acquire a shared lock for $r_1(x)$ and $r_2(x)$. Now, when $w_1(x)$ comes, T_1 waits for T_2 to release the lock. When $w_2(x)$ comes, T_2 waits for T_1 to release the lock, causing deadlock. Hence, giving s_1 to 2PL scheduler causes deadlock.

O2PL:

Here, all the transactions are able to acquire the lock, but during unlocking, T_1 waits for T_2 because of $\{r_2(x), w_1(x)\}$, and T_2 waits for T_1 because of $\{w_1(x), w_2(x)\}$, causing deadlock. Hence, giving s_1 to O2PL scheduler causes deadlock.

BTO:

The input schedule runs as usual until $w_1(x)$ happens. At that point, we see that $w_4(x)$ happens before $w_1(x)$ in s_2, but T_4 has a later timestamp than T_1 (since it starts later). Because of this, T_1 has to be aborted. Similarly, $w_2(x)$ also happens too late, so T_2 must be aborted as well. The resulting output schedule is:

$r_1(x)r_2(x)w_3(x)w_4(x)a_1a_2c_3c_4$

SGT:

The input schedule s_2 runs normally without changes until $w_1(x)$ happens, since no cycles have formed in the serialization graph yet. However, when $w_1(x)$ arrives, multiple cycles appear in the serialization graph. Because of this, the scheduler aborts T_1. Later, when $w_2(x)$ occurs, it adds a new edge to the serialization graph, forming a cycle $T_3 \rightarrow T_2 \rightarrow T_3$. As a result, T_2 is also aborted. The resulting output schedule is:

$r_1(x)r_2(x)w_3(x)w_4(x)a_1a_2c_3c_4$

Q2 (4.9)

Consider the following schedule:

$s = w_1(x)w_2(x)w_2(y)w_1(y)$

On giving this schedule to O2PL scheduler, both T_1 and T_2 obtain locks for all the operations. But during unlocking, T_2 waits for T_1 because of $\{w_1(x),\ w_2(x)\}$ and T_1 waits for T_2 because of $\{w_2(y),\ w_1(y)\}$ causing deadlock.

Hence, O2PL is susceptible to deadlocks.

Q3 (4.12)

Consider the following history:

$s = w_2(x)w_1(x)w_1(z)c_1w_3(y)w_2(y)c_2w_3(z)c_3$

And the following prefix:

$s' = w_2(x)w_1(x)w_1(z)c_1w_3(y)w_2(y)c_2$

After executing s', the graph is as follows: $T_1 <- T_2 <- T_3$

Now, c_2 commits and hence, T_1 is removed from the graph since no transaction active during T_1 is active anymore.

But when $w_3(z)$ arrives, it adds the edge $T_1 -> T_3$ because of $\{w_1(z),\ w_3(z)\}$ and causes the cycle: $T_1 <- T_2 <- T_3 <- T_1$

But since T_1 is already removed from the graph, the cycle will not be detected.

Hence, the given condition leads to incorrect behavior of the SGT protocol.

Q4 (4.16)

$s = r_1(x)r_2(x)r_1(y)r_3(x)w_1(x)w_1(y)c_1r_2(y)r_3(z)w_3(z)c_3r_2(z)c_2$

BOCC:

T_1 executes normally. When T_2 enters it's validation phase, it sees that $RS(t_2) \cap WS(t_1) \neq \emptyset$ because of $\{w_1(x),\ r_2(x)\}$. Hence, T_2 is aborted. Similarly, when T_3 enters it's validation phase, it sees that $RS(t_3) \cap WS(t_1) \neq \emptyset$ because of $\{w_1(x),\ r_3(x)\}$. Hence, T_3 is aborted. Final output:

$r_1(x)r_2(x)r_1(y)r_3(x)w_1(x)w_1(y)c_1r_2(y)r_3(z)a_3r_2(z)a_2$

FOCC:

When T_1 enters it's validation phase, it sees that $WS(t_1) \cap RS(t_2,\ t_3) \neq \emptyset$ because of $\{w_1(x),\ r_2(x)\}$ and $\{w_1(x),\ r_3(x)\}$ and hence, aborts. T_2 and T_3 execute normally. Hence, the final output:

$r_1(x)r_2(x)r_1(y)r_3(x)a_1r_2(y)r_3(z)w_3(z)c_3r_2(z)c_2$